

```
import nltk # Import Natural Language Toolkit library
import re # Import regular expression module for text cleaning
import math # Import math module for mathematical operations like log and exp
from collections import Counter, defaultdict # Import Counter for counting frequencies and defaultdict for handling missing
```

```
paragraph = """
Natural language processing is an important area of artificial intelligence.
It helps computers understand and interpret human language effectively.
Language models are widely used in modern NLP applications.
They predict the next word based on previous context.
Unigram models consider each word independently.
Bigram models capture relationships between consecutive words.
Trigram models use longer context for better prediction.
Smoothing techniques handle unseen words in datasets.
Perplexity is used to evaluate language model performance.
Lower perplexity indicates a better predictive model.
"""
# Define a sample paragraph of text to be used as a corpus for language modeling
```

```
# Multiply paragraph 40 times to create a larger corpus for training
corpus = paragraph * 40

print("Total words:", len(corpus.split())) # Print the total number of words in the corpus
print(corpus[:600]) # Print the first 600 characters of the corpus to display a sample
```

Total words: 3200

Natural language processing is an important area of artificial intelligence.
It helps computers understand and interpret human language effectively.
Language models are widely used in modern NLP applications.
They predict the next word based on previous context.
Unigram models consider each word independently.
Bigram models capture relationships between consecutive words.
Trigram models use longer context for better prediction.
Smoothing techniques handle unseen words in datasets.
Perplexity is used to evaluate language model performance.
Lower perplexity indicates a better predictive model.

----- STEP 4: PREPROCESS TEXT -----

```
def preprocess_text(text):
    # Convert text to lowercase to ensure consistency
    text = text.lower()

    # Remove punctuation and numbers using a regular expression, keeping only letters and spaces
    text = re.sub(r'^a-z\s', '', text)

    # Split text into sentences based on the period ('.') delimiter
    sentences = text.split('.')

    tokens = [] # Initialize an empty list to store processed tokens
    for sentence in sentences:
        words = sentence.split() # Split each sentence into individual words
        if words:
            tokens.append('<s>') # Add a start-of-sentence token
            tokens.extend(words) # Add the words from the sentence
            tokens.append('</s>') # Add an end-of-sentence token
    return tokens # Return the list of preprocessed tokens
```

```
# Apply the preprocessing function to the entire corpus
tokens = preprocess_text(corpus)

# ----- PRINT STEP 4 OUTPUT -----
print("STEP 4 OUTPUT: Preprocessed Tokens (First 50)") # Informative header for the output
print(tokens[:50]) # Print the first 50 preprocessed tokens to show a sample
print("\nTotal number of tokens:", len(tokens)) # Print the total count of tokens in the corpus
```

STEP 4 OUTPUT: Preprocessed Tokens (First 50)
['<s>', 'natural', 'language', 'processing', 'is', 'an', 'important', 'area', 'of', 'artificial', 'intelligence', 'it', 'hel
Total number of tokens: 3202

----- STEP 5: BUILD N-GRAM MODELS -----

```
# Unigram model: Count the frequency of each individual token
unigram_counts = Counter(tokens)
total_unigrams = sum(unigram_counts.values()) # Total number of unigrams (tokens)
```

```
vocab_size = len(unigram_counts) # Size of the vocabulary (number of unique unigrams)
```

```
# Bigram model: Count the frequency of consecutive pairs of tokens
```

```
bigram_counts = Counter(zip(tokens[:-1], tokens[1:]))
```

```
# Trigram model: Count the frequency of consecutive triplets of tokens
```

```
trigram_counts = Counter(zip(tokens[:-2], tokens[1:-1], tokens[2:]))
```

```
print("\nSTEP 5 OUTPUT: Top 10 Unigram Counts") # Header for unigram counts
```

```
for word, count in unigram_counts.most_common(10): # Iterate through the top 10 most common unigrams
```

```
    print(word, ":", count) # Print each unigram and its count
```

```
print("\nTop 10 Bigram Counts") # Header for bigram counts
```

```
for bigram, count in bigram_counts.most_common(10): # Iterate through the top 10 most common bigrams
```

```
    print(bigram, ":", count) # Print each bigram and its count
```

```
print("\nTop 10 Trigram Counts") # Header for trigram counts
```

```
for trigram, count in trigram_counts.most_common(10): # Iterate through the top 10 most common trigrams
```

```
    print(trigram, ":", count) # Print each trigram and its count
```

```
STEP 5 OUTPUT: Top 10 Unigram Counts
```

```
language : 160
```

```
models : 160
```

```
is : 80
```

```
used : 80
```

```
in : 80
```

```
word : 80
```

```
context : 80
```

```
words : 80
```

```
better : 80
```

```
perplexity : 80
```

```
Top 10 Bigram Counts
```

```
('natural', 'language') : 40
```

```
('language', 'processing') : 40
```

```
('processing', 'is') : 40
```

```
('is', 'an') : 40
```

```
('an', 'important') : 40
```

```
('important', 'area') : 40
```

```
('area', 'of') : 40
```

```
('of', 'artificial') : 40
```

```
('artificial', 'intelligence') : 40
```

```
('intelligence', 'it') : 40
```

```
Top 10 Trigram Counts
```

```
('natural', 'language', 'processing') : 40
```

```
('language', 'processing', 'is') : 40
```

```
('processing', 'is', 'an') : 40
```

```
('is', 'an', 'important') : 40
```

```
('an', 'important', 'area') : 40
```

```
('important', 'area', 'of') : 40
```

```
('area', 'of', 'artificial') : 40
```

```
('of', 'artificial', 'intelligence') : 40
```

```
('artificial', 'intelligence', 'it') : 40
```

```
('intelligence', 'it', 'helps') : 40
```

```
def unigram_probability(sentence):
```

```
    words = preprocess_text(sentence) # Preprocess the input sentence
```

```
    prob = 1.0 # Initialize probability to 1.0
```

```
    for w in words:
```

```
        # Calculate probability using Laplace smoothing (add-1 smoothing)
```

```
        prob *= (unigram_counts[w] + 1) / (total_unigrams + vocab_size)
```

```
    return prob # Return the calculated unigram probability for the sentence
```

```
def bigram_probability(sentence):
```

```
    words = preprocess_text(sentence) # Preprocess the input sentence
```

```
    prob = 1.0 # Initialize probability to 1.0
```

```
    for i in range(len(words) - 1):
```

```
        # Calculate bigram probability using Laplace smoothing
```

```
        #  $P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + 1}{\text{count}(w_{i-1}) + \text{vocab\_size}}$ 
```

```
        prob *= (bigram_counts[(words[i], words[i+1])] + 1) / \
                (unigram_counts[words[i]] + vocab_size)
```

```
    return prob # Return the calculated bigram probability for the sentence
```

```
def trigram_probability(sentence):
```

```
    words = preprocess_text(sentence) # Preprocess the input sentence
```

```
    prob = 1.0 # Initialize probability to 1.0
```

```
    for i in range(len(words) - 2):
```

```
        # Calculate trigram probability using Laplace smoothing
```

```
        #  $P(w_i | w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i) + 1}{\text{count}(w_{i-2}, w_{i-1}) + \text{vocab\_size}}$ 
```

```
        prob *= (trigram_counts[(words[i], words[i+1], words[i+2])] + 1) / \
                (bigram_counts[(words[i], words[i+1])] + vocab_size)
```

```
    return prob # Return the calculated trigram probability for the sentence
```

```
test_sentences = [ # Define a list of sentences to test the language models
    "natural language processing is important",
    "language models predict the next word",
    "bigram models capture word relationships",
    "smoothing handles unseen words",
    "lower perplexity means better model"
]
```

```
print("STEP 7 OUTPUT: SENTENCE PROBABILITIES\n") # Informative header for the output
```

```
for s in test_sentences: # Iterate through each test sentence
    print("Sentence:", s) # Print the current sentence
    print("Unigram Probability :", unigram_probability(s)) # Calculate and print unigram probability
    print("Bigram Probability :", bigram_probability(s)) # Calculate and print bigram probability
    print("Trigram Probability :", trigram_probability(s)) # Calculate and print trigram probability
    print("-" * 50) # Print a separator line for readability
```

```
STEP 7 OUTPUT: SENTENCE PROBABILITIES
```

```
Sentence: natural language processing is important
Unigram Probability : 9.011934205024813e-16
Bigram Probability : 4.95881753346223e-08
Trigram Probability : 6.02369065431748e-07
```

```
-----
Sentence: language models predict the next word
Unigram Probability : 4.4384258397338466e-17
Bigram Probability : 4.478227287928446e-09
Trigram Probability : 2.856820787373334e-09
```

```
-----
Sentence: bigram models capture word relationships
Unigram Probability : 9.011934205024813e-16
Bigram Probability : 6.047338455441745e-10
Trigram Probability : 1.1906713212463524e-08
```

```
-----
Sentence: smoothing handles unseen words
Unigram Probability : 4.462962114259371e-16
Bigram Probability : 5.347062386029288e-09
Trigram Probability : 3.107361740813652e-08
```

```
-----
Sentence: lower perplexity means better model
Unigram Probability : 2.184713340520017e-17
Bigram Probability : 5.2953461548904055e-11
Trigram Probability : 4.6378533444979876e-10
-----
```

```
def calculate_perplexity(sentence, model="unigram"):
    words = preprocess_text(sentence) # Preprocess the input sentence
    N = len(words) # Get the number of words (tokens) in the preprocessed sentence
    log_prob = 0.0 # Initialize log probability to 0.0

    if model == "unigram":
        for w in words:
            # Calculate unigram probability with Laplace smoothing and add its log to log_prob
            p = (unigram_counts[w] + 1) / (total_unigrams + vocab_size)
            log_prob += math.log(p)

    elif model == "bigram":
        for i in range(len(words) - 1):
            # Calculate bigram probability with Laplace smoothing and add its log to log_prob
            p = (bigram_counts[(words[i], words[i+1])] + 1) / \
                (unigram_counts[words[i]] + vocab_size)
            log_prob += math.log(p)

    else: # trigram model
        for i in range(len(words) - 2):
            # Calculate trigram probability with Laplace smoothing and add its log to log_prob
            p = (trigram_counts[(words[i], words[i+1], words[i+2])] + 1) / \
                (bigram_counts[(words[i], words[i+1])] + vocab_size)
            log_prob += math.log(p)

    # Calculate perplexity: exp(-(1/N) * log_prob)
    return math.exp(-log_prob / N)
```

```
print("\nSTEP 8 OUTPUT: PERPLEXITY VALUES\n") # Informative header for the output
```

```
for s in test_sentences: # Iterate through each test sentence
    print("Sentence:", s) # Print the current sentence
    print("Unigram Perplexity :", calculate_perplexity(s, "unigram")) # Calculate and print unigram perplexity
    print("Bigram Perplexity :", calculate_perplexity(s, "bigram")) # Calculate and print bigram perplexity
    print("Trigram Perplexity :", calculate_perplexity(s, "trigram")) # Calculate and print trigram perplexity
    print("-" * 50) # Print a separator line for readability
```

STEP 8 OUTPUT: PERPLEXITY VALUES

Sentence: natural language processing is important
Unigram Perplexity : 141.03006709505294
Bigram Perplexity : 11.053947822513752
Trigram Perplexity : 7.73732872449535

Sentence: language models predict the next word
Unigram Perplexity : 110.68693941869675
Bigram Perplexity : 11.056348860009795
Trigram Perplexity : 11.695387758439026

Sentence: bigram models capture word relationships
Unigram Perplexity : 141.03006709505294
Bigram Perplexity : 20.745284217099186
Trigram Perplexity : 13.552821847286495

Sentence: smoothing handles unseen words
Unigram Perplexity : 361.73966603986315
Bigram Perplexity : 23.91373752369205
Trigram Perplexity : 17.83479127158678

Sentence: lower perplexity means better model
Unigram Perplexity : 239.93107818963037
Bigram Perplexity : 29.377517397860995
Trigram Perplexity : 21.546824983575245
